

Compte Rendu - TP Théorie de l'Information et Codage Source

Module: Communications numériques & codage de l'information
Filière: SICOM - S6

1. Introduction

Ce TP est une introduction aux algorithmes de compression sans pertes (lossless). Ces méthodes sont essentielles et forment la base des logiciels de compression tels que gzip, zip, compress, etc. On les retrouve également dans les méthodes de compression d'images (formats GIF, PNG, JPEG).

2. Compression sans perte

2.1 Le codage RLE (Run Length Encoding)

Le codage RLE exploite la succession des symboles identiques. Son principe est basé sur 3 règles:

Règle 1: Codage de valeurs redondantes - Une suite de N valeurs identiques successives est remplacée par le couple (N, V) où N représente le nombre de répétitions et V la valeur. Condition: $N \geq 3$.

Exemple: 23 23 23 23 23 23 23 23 23 23 23 23 23 23 $\rightarrow$ (14, 23)

Règle 2: Codage d'une suite paire sans répétition - Écrire: 0 + nombre d'éléments + les valeurs une à une

Exemple: 25 3 0 5 44 21 87 8 6 3 4 2 55 9 $\rightarrow$ 0 14 25 3 0 5 44 21 87 8 6 3 4 2 55 9

Règle 3: Codage d'une suite impaire sans répétition - Ajouter un 0 à la fin si la chaîne contient un nombre impair de valeurs.

Exemple: 25 4 9 68 23 54 8 9 6 $\rightarrow$ 0 9 25 4 9 68 23 54 8 9 6 0

2.2 Mise en œuvre du codage RLE

Suite de départ: 128 233 225 3 3 3 3 3 3 3 3 3 3 3 2 3 5 6

Partie	Type	Codage
128 233 225	Impaire (3 éléments)	0 3 128 233 225 0
13 $\times$ 3	Répétition	13 3
2 3 5 6	Paire (4 éléments)	0 4 2 3 5 6

Résultat: 0 3 128 233 225 0 13 3 0 4 2 3 5 6 (14 valeurs au lieu de 20 $\rightarrow$ gain de 6 emplacements)

2.3 Avantages et Inconvénients du RLE

Avantages: Très efficace pour les données avec beaucoup de répétitions. Exemple: compression JPEG de 64 coefficients en 10 éléments.

Inconvénients: Si pas de redondance, peut augmenter la taille. Limite de 255 répétitions sur 8 bits.

3. Information propre et Entropie

3.1 Information propre (Self-information)

La quantité d'information véhiculée par un message n d'une source discrète:

$$I(n) = -\log_2(p)$$

où p est la probabilité d'apparition du message. Plus la probabilité est faible, plus l'information est grande.

3.2 Entropie d'une source

L'entropie H est l'espérance mathématique des quantités d'information:

$$H = \sum p \times \log_2(1/p) = -\sum p \times \log_2(p)$$

Unité: Shannon (bit d'information).

3.3 Entropie maximale

Quand tous les symboles sont équiprobables: $H_{\max} = \log_2(m)$ où m est le nombre total de symboles.

4. Codage Arithmétique

4.1 Principe

Le codage arithmétique affecte à l'ensemble du message un seul nombre en virgule flottante.

Étapes:

1. Calculer la probabilité de chaque symbole
2. Associer à chaque symbole un sous-intervalle proportionnel à sa probabilité dans $[0,1]$
3. Répéter pour chaque symbole: réduire l'intervalle de travail
4. La borne inférieure code le message de manière unique

4.2 Exemple: Codage de 'BILL GATES'

Probabilités (fréquences relatives):

Symbole	Espace	A	B	E	G	I	L	S	T
Probabilité	1/10	1/10	1/10	1/10	1/10	1/10	2/10	1/10	1/10

4.3 Progression du codage

Symbole	Intervalle
B	[0.2, 0.3[
BI	[0.25, 0.26[
BIL	[0.256, 0.258[
Final	[0.2572167752, 0.2572167756[

Résultat final: borne inférieure = 0.2572167752

5. Codage de Huffman

5.1 Principe

Le code de Huffman est un code préfixé (aucun mot n'est le début d'un autre). Il est uniquement déchiffrable.

5.2 Construction de l'arbre

1. Classer les symboles par probabilités croissantes
2. Fusionner les deux symboles de probabilités les plus faibles en un nœud parent
3. Attribuer au nœud parent la somme des probabilités
4. Répéter jusqu'à un seul nœud restant

5.3 Exemple: Symboles A, B, C, D, E

Symbole	A	B	C	D	E
Probabilité	0.4	0.2	0.2	0.1	0.1

Construction:

- $E + D \rightarrow I (0.2) \rightarrow I + C \rightarrow II (0.4) \rightarrow II + B \rightarrow III (0.6) \rightarrow III + A \rightarrow IV (1.0)$

5.4 Codes résultants

Symbole	A	B	C	D	E
Code	1	000	001	010	011

Longueur moyenne: $L = 0.4 \times 1 + 0.2 \times 3 + 0.2 \times 3 + 0.1 \times 3 + 0.1 \times 3 = 2.2$ bits/symbole

6. Codage LZW (Lempel-Ziv-Welch)

6.1 Principe

Construire un dictionnaire des séquences redondantes et remplacer chaque séquence par son adresse.

6.2 Caractéristiques

- Le dictionnaire est construit pendant le traitement
- Il n'est pas sauvegardé avec le fichier compressé
- Les séquences ont une taille variable

6.3 Algorithme de compression

1. Lire le premier octet c
2. Tant que non EOF: $s :=$ octet suivant, $seq := c \mid s$
 - si seq est dans le dictionnaire, alors $c := seq$
 - sinon écrire $adresse(c)$, ajouter seq au dictionnaire, $c := s$
3. Écrire $adresse(c)$

7. Tableau comparatif des méthodes

Méthode	Type	Avantage	Inconvénient
RLE	Par longueur	Simple, rapide	Inefficace sans redondance
Arithmétique	Entropique	Compression proche entropie	Complexité de calcul
Huffman	Arbre préfixé	Optimal pour VLC	Nécessite le dictionnaire
LZW	Dictionnaire	Pas de transmission dict	Construction temps réel

8. Applications pratiques

Standards utilisant ces méthodes:

- JPEG: Codage de Huffman (après DCT)
- PNG/GIF: LZW pour compression sans perte
- GZIP/ZIP: LZ77 + Huffman
- JPEG2000: Arithmétique (EBCOT)

Critère d'efficacité: Taux de compression = Taille originale / Taille compressée

9. Conclusion

Les codeurs entropiques (Huffman, arithmétique) et les méthodes par dictionnaire (LZW) sont fondamentaux en compression de données. Le choix de la méthode dépend du type de données, du taux de compression souhaité, de la complexité de calcul acceptable, et de la nécessité ou non de transmettre le dictionnaire. L'entropie de la source reste la limite théorique inférieure impossible à atteindre en codage sans perte.

Annexe

Les programmes MATLAB lz78_codage.m et lz78_decodage.m permettent de mettre en œuvre l'algorithme LZW pour la compression et décompression de données.